

Phase 1 Structured Note: Graph Fundamentals and Terminology

1. Current Learning Position

You are at the beginning of the graph curriculum.

Before learning algorithms like **BFS**, **DFS**, or **Disjoint Sets**, you must first understand the basic language of graphs.

In this phase, the goal is to become comfortable with words like:

- graph
- vertex
- edge
- directed graph
- undirected graph
- degree
- indegree
- outdegree
- path
- cycle
- connected graph
- DAG
- topological ordering

Think of Phase 1 as learning the alphabet before writing full sentences. Later phases will use these terms again and again.

2. What Is a Graph?

A **graph** is a data structure used to represent **objects** and the **relationships** between those objects.

A graph is written mathematically as:

$$G = (V, E)$$

This means:

Graph = Vertices + Edges

Main parts of a graph

Term	Simple meaning
Vertex	A node, point, item, or object in a graph
Edge	A connection or relationship between two vertices
V	The set of all vertices
E	The set of all edges

Simple example

```
1 ---- 2
|       |
3 ---- 4
```

In this graph:

```
Vertices = {1, 2, 3, 4}
Edges = {1--2, 1--3, 2--4, 3--4}
```

Beginner intuition

An **array** is good when data is arranged in a straight line.

Example:

```
10, 20, 30, 40
```

A **graph** is better when every item may connect to many other items.

Example:

```
1 connects to 2 and 3
2 connects to 1 and 4
```

3 connects to 1 and 4
4 connects to 2 and 3

Graphs are useful for network-like problems.

Real-world examples of graphs

Real-world system	Vertices	Edges
Social network	People	Friendships or follows
Road map	Cities	Roads
Course schedule	Courses	Prerequisite relationships
Computer network	Computers	Connections between computers
Web pages	Pages	Links between pages

3. Directed and Undirected Graphs

Graphs can have different types of edges. The two main types are:

1. **Undirected edges**
2. **Directed edges**

3.1 Undirected Graph

An **undirected graph** has edges that work both ways.

Example:

1 ---- 2

This means:

1 is connected to 2
2 is connected to 1

There is no arrow. You can travel in both directions.

Real-world example

Friendship is often undirected.

If Ali is friends with Sara, then Sara is also friends with Ali.

Ali ---- Sara

3.2 Directed Graph

A **directed graph** has edges with arrows.

A directed graph is also called a **digraph**.

Example:

1 ---> 2

This means:

1 points to 2

But it does **not** automatically mean:

2 points to 1

The direction matters.

Real-world example

Following someone on social media is directed.

If Ali follows Sara, Sara does not automatically follow Ali.

Ali ---> Sara

4. Incoming and Outgoing Edges

Incoming and outgoing edges are used in **directed graphs**.

Look at this edge:

1 ---> 2

For vertex 1:

The edge is leaving 1.

So it is an **outgoing edge** from vertex 1.

For vertex 2:

The edge is entering 2.

So it is an **incoming edge** to vertex 2.

Important terms

Term	Simple meaning
Outgoing edge	An edge that leaves a vertex
Incoming edge	An edge that enters a vertex
Outdegree	Number of edges leaving a vertex
Indegree	Number of edges entering a vertex

4.1 Indegree

Indegree means the number of arrows entering a vertex.

Example:

```
1 ---> 3
2 ---> 3
4 ---> 3
```

Vertex **3** has three arrows entering it.

So:

```
indegree(3) = 3
```

4.2 Outdegree

Outdegree means the number of arrows leaving a vertex.

Example:

```
1 ---> 2
1 ---> 3
1 ---> 4
```

Vertex **1** has three arrows leaving it.

So:

```
outdegree(1) = 3
```

4.3 Indegree and Outdegree Example

Example graph:

```
1 ---> 2
1 ---> 3
4 ---> 1
```

For vertex **1**:

```
Outgoing edges: 1 -> 2, 1 -> 3
Incoming edges: 4 -> 1
```

So:

```
outdegree(1) = 2
indegree(1) = 1
```

For vertex **2**:

```
Incoming edges: 1 -> 2
Outgoing edges: none
```

So:

```
indegree(2) = 1
outdegree(2) = 0
```

Easy memory trick

```
In-degree = arrows going in
Out-degree = arrows going out
```

5. Degree in an Undirected Graph

In an **undirected graph**, we usually do not say indegree or outdegree because there are no arrows.

Instead, we simply say **degree**.

The **degree** of a vertex is the number of edges touching that vertex.

Example:

```
1 ---- 2
|
3
```

Vertex **1** touches two edges:

1--2
1--3

So:

$\text{degree}(1) = 2$

Vertex **2** touches one edge:

$\text{degree}(2) = 1$

Vertex **3** touches one edge:

$\text{degree}(3) = 1$

Beginner rule

In an undirected graph, degree = number of edges attached to the vertex.

6. Adjacent Vertices

Two vertices are **adjacent** if they are directly connected by an edge.

Example:

1 ---- 2 ---- 3

Here:

1 and 2 are adjacent.
2 and 3 are adjacent.
1 and 3 are not adjacent.

Why are **1** and **3** not adjacent?

Because there is no direct edge between them.

Even though you can go from 1 to 3 through 2, they are not directly connected.

Important idea

Adjacent means directly connected, not indirectly reachable.

7. Self-Loops, Parallel Edges, and Simple Graphs

Some graphs may have special edge cases.

The important ones in Phase 1 are:

1. Self-loop
2. Parallel edge
3. Simple graph

7.1 Self-Loop

A **self-loop** is an edge from a vertex back to itself.

Example:

3 ---> 3

This edge starts at vertex 3 and ends at vertex 3.

So it is a self-loop.

Simple meaning

A self-loop means a vertex is connected to itself.

7.2 Parallel Edges

Parallel edges are multiple edges connecting the same vertices.

Example:

```
1 ==== 2
```

This means there is more than one edge between vertex **1** and vertex **2**.

Another way to imagine it:

```
1 ---- 2  
1 ---- 2
```

The edge appears more than once.

7.3 Simple Graph

A **simple graph** is a graph that has:

```
No self-loops  
No parallel edges
```

Example of a simple graph

```
1 ---- 2 ---- 3
```

This is simple because:

```
No vertex connects to itself.  
No repeated edge exists between the same two vertices.
```

Example of a graph that is not simple

```
1 ---- 2  
1 ---- 2  
3 ---- 3
```

This is not simple because:

1--2 appears more than once, so it has parallel edges.
3--3 is a self-loop.

Why this matters

Many beginner graph algorithms first assume the graph is simple.

Self-loops and parallel edges can make some tasks more careful, such as:

- counting degrees
- detecting cycles
- checking if a graph is valid

8. Connectivity Concepts

Connectivity is about whether vertices can reach each other.

The main connectivity concepts are:

1. Connected graph
2. Non-connected graph
3. Connected component
4. Articulation point
5. Biconnected graph
6. Strongly connected directed graph

8.1 Connected Graph

An undirected graph is **connected** if every vertex can reach every other vertex.

Example:

1 ---- 2 ---- 3 ---- 4

This graph is connected because:

From 1, you can reach 2, 3, and 4.
From 2, you can reach 1, 3, and 4.

From 3, you can reach 1, 2, and 4.
From 4, you can reach 1, 2, and 3.

You may need to pass through other vertices, and that is okay.

Simple meaning

Connected means the graph is all in one piece.

8.2 Non-Connected Graph

A graph is **non-connected** if it is broken into separate pieces.

Example:

1 ---- 2 3 ---- 4

Here:

1 can reach 2.
3 can reach 4.
But 1 cannot reach 3 or 4.

So the graph is not connected.

Simple meaning

Non-connected means the graph has separate groups.

8.3 Connected Components

A **connected component** is one connected piece of a graph.

Example:

1 ---- 2 3 ---- 4 5

This graph has three connected components:

Component 1: {1, 2}
Component 2: {3, 4}
Component 3: {5}

Vertex 5 is alone, but it still counts as a connected component.

Simple meaning

A connected component is one separate connected group inside a graph.

8.4 Articulation Point

An **articulation point** is a vertex that breaks the graph into more pieces if it is removed.

Example:

1 ---- 2 ---- 3

If we remove vertex 2, the graph becomes:

1 3

Now 1 and 3 are separated.

So:

Vertex 2 is an articulation point.

Simple meaning

An articulation point is an important bridge-like vertex.
If you remove it, the graph becomes more disconnected.

8.5 Biconnected Graph

A **biconnected graph** is a connected graph with no articulation point.

That means:

The graph is connected.
Removing any one vertex does not break it apart.

Example:

```
1 ---- 2
|      |
4 ---- 3
```

This graph is like a square.

If you remove one vertex, the remaining vertices are still connected.

Simple meaning

A biconnected graph is strongly held together.
It does not depend on one single vertex to stay connected.

8.6 Strongly Connected Directed Graph

For directed graphs, connectivity is stricter because arrows must be followed.

A directed graph is **strongly connected** if every vertex can reach every other vertex while following arrow directions.

Example:

```
1 ---> 2
^       |
|       v
4 <--- 3
```

This forms a directed cycle:

```
1 -> 2 -> 3 -> 4 -> 1
```

From any vertex, you can reach every other vertex by following arrows.

So this graph is strongly connected.

Not strongly connected example

```
1 ----> 2 ----> 3
```

This graph is not strongly connected because:

```
1 can reach 2 and 3.  
2 can reach 3.  
But 3 cannot go back to 2 or 1.
```

Important difference

Type of graph	Connectivity idea
Undirected graph	Connected means every vertex can reach every other vertex
Directed graph	Strongly connected means every vertex can reach every other vertex while following arrows

9. Paths and Cycles

Paths and cycles are very important in graph algorithms.

9.1 Path

A **path** is a sequence of vertices where each neighboring pair is connected by an edge.

Example:

```
1 ---- 2 ---- 3 ---- 4
```

A path from 1 to 4 is:

```
1 -> 2 -> 3 -> 4
```

This is a path because:

```
1 is connected to 2.  
2 is connected to 3.  
3 is connected to 4.
```

Simple meaning

A path is a valid route through the graph.

9.2 Cycle or Circuit

A **cycle** is a path that starts and ends at the same vertex.

A cycle may also be called a **circuit**.

Example:

```
1 ---- 2  
|      |  
4 ---- 3
```

One cycle is:

```
1 -> 2 -> 3 -> 4 -> 1
```

This is a cycle because it starts at 1 and returns to 1.

Simple meaning

A cycle is a route that comes back to where it started.

Important rule

Every cycle is a path.
But not every path is a cycle.

Why?

Because a normal path does not have to return to the starting point.

10. DAG: Directed Acyclic Graph

A **DAG** means:

Directed Acyclic Graph

Break the word down:

Part	Meaning
Directed	The graph has arrows
Acyclic	The graph has no cycles
Graph	A set of vertices and edges

So a DAG is:

A graph with arrows and no cycles.

10.1 DAG Example

Math ---> Programming ---> Data Structures

This can be a DAG because:

The edges have direction.
There is no way to follow arrows and return to the start.

10.2 Not a DAG Example

```
1 ---> 2
^       |
|       v
4 <--- 3
```

This is not a DAG because it has a cycle:

```
1 -> 2 -> 3 -> 4 -> 1
```

Even though it is directed, it is not acyclic.

So it is not a DAG.

11. Topological Ordering

A **topological ordering** is a linear order of vertices in a DAG.

It arranges vertices so that every directed edge points forward in the order.

Simple example

```
Math ---> Programming ---> Data Structures
```

A valid topological order is:

```
Math, Programming, Data Structures
```

Why?

Because:

Math must come before Programming.
Programming must come before Data Structures.

11.1 Real-world use: Course prerequisites

Suppose:

Math must be completed before Programming.
Programming must be completed before Data Structures.

Then the graph is:

Math ---> Programming ---> Data Structures

The correct study order is:

Math first
Programming second
Data Structures third

This is topological ordering.

11.2 Why topological ordering only works for DAGs

Topological ordering only works if there is no cycle.

Example with a cycle:

A ---> B
B ---> C
C ---> A

This means:

A must come before B.
B must come before C.
C must come before A.

This is impossible because **A** must come before itself indirectly.

So:

If a directed graph has a cycle, no valid topological ordering exists.

12. Phase 1 C++ Program: Counting Degrees

This program shows how to calculate:

- indegree
- outdegree
- degree

It uses simple C++ vectors and pairs.

12.1 Full Code

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n = 4;

    // Directed edges: 1 -> 2, 1 -> 3, 3 -> 2, 4 -> 1
    vector<pair<int, int>> directedEdges = {
        {1, 2}, {1, 3}, {3, 2}, {4, 1}
    };

    vector<int> indegree(n + 1, 0);
    vector<int> outdegree(n + 1, 0);

    for (auto edge : directedEdges) {
        int u = edge.first;
        int v = edge.second;
```

```

        outdegree[u]++;
        indegree[v]++;
    }

    cout << "Directed graph:\n";
    for (int v = 1; v <= n; v++) {
        cout << "Vertex " << v
            << ": indegree = " << indegree[v]
            << ", outdegree = " << outdegree[v] << endl;
    }

    // Undirected edges: 1--2, 1--3, 2--4
    vector<pair<int, int>> undirectedEdges = {
        {1, 2}, {1, 3}, {2, 4}
    };

    vector<int> degree(n + 1, 0);

    for (auto edge : undirectedEdges) {
        int u = edge.first;
        int v = edge.second;

        degree[u]++;
        degree[v]++;
    }

    cout << "\nUndirected graph:\n";
    for (int v = 1; v <= n; v++) {
        cout << "Vertex " << v
            << ": degree = " << degree[v] << endl;
    }

    return 0;
}

```

13. Code Explanation in Simple Words

13.1 Number of vertices

```
int n = 4;
```

This means the graph has 4 vertices:

```
1, 2, 3, 4
```

The code uses vertex numbers starting from `1`.

13.2 Directed edges

```
vector<pair<int, int>> directedEdges = {  
    {1, 2}, {1, 3}, {3, 2}, {4, 1}  
};
```

Each pair stores one directed edge.

For example:

```
{1, 2}
```

means:

```
1 -> 2
```

So the directed graph has these edges:

```
1 -> 2  
1 -> 3  
3 -> 2  
4 -> 1
```

13.3 Indegree and outdegree arrays

```
vector<int> indegree(n + 1, 0);  
vector<int> outdegree(n + 1, 0);
```

These arrays store counts.

```
indegree[v] = number of arrows entering vertex v
outdegree[v] = number of arrows leaving vertex v
```

Why `n + 1`?

Because the vertices are numbered from `1` to `n`.

So index `0` is ignored.

13.4 Updating indegree and outdegree

```
for (auto edge : directedEdges) {
    int u = edge.first;
    int v = edge.second;

    outdegree[u]++;
    indegree[v]++;
}
```

For every directed edge:

```
u -> v
```

The edge leaves `u`, so:

```
outdegree[u]++;
```

The edge enters `v`, so:

```
indegree[v]++;
```

13.5 Undirected edges

```
vector<pair<int, int>> undirectedEdges = {
    {1, 2}, {1, 3}, {2, 4}
};
```

These edges are undirected:

```
1--2
1--3
2--4
```

There are no arrows.

13.6 Degree array

```
vector<int> degree(n + 1, 0);
```

This array stores the degree of each vertex.

```
degree[v] = number of edges touching vertex v
```

13.7 Updating degree

```
for (auto edge : undirectedEdges) {
    int u = edge.first;
    int v = edge.second;

    degree[u]++;
    degree[v]++;
}
```

For an undirected edge:

```
u--v
```

Both `u` and `v` touch the edge.

So both degrees increase.

14. Dry Run of the C++ Program

14.1 Directed graph dry run

Directed edges:

```
1 -> 2
1 -> 3
3 -> 2
4 -> 1
```

Start with all values as 0:

```
indegree: all 0
outdegree: all 0
```

Edge 1: 1 -> 2

The edge leaves 1 and enters 2.

```
outdegree[1] increases by 1
indegree[2] increases by 1
```

Edge 2: 1 -> 3

The edge leaves 1 and enters 3.

```
outdegree[1] increases by 1 again
indegree[3] increases by 1
```

Now vertex 1 has two outgoing edges.

Edge 3: 3 -> 2

The edge leaves 3 and enters 2.

outdegree[3] increases by 1
indegree[2] increases by 1 again

Now vertex 2 has two incoming edges.

Edge 4: 4 -> 1

The edge leaves 4 and enters 1.

outdegree[4] increases by 1
indegree[1] increases by 1

14.2 Final directed graph result

Vertex	Indegree	Outdegree	Explanation
1	1	2	Receives from 4; points to 2 and 3
2	2	0	Receives from 1 and 3; points to nobody
3	1	1	Receives from 1; points to 2
4	0	1	Receives from nobody; points to 1

14.3 Undirected graph dry run

Undirected edges:

1--2
1--3
2--4

Start with all degrees as 0.

Edge 1: 1--2

Both vertices touch the edge.

degree[1] increases by 1
degree[2] increases by 1

Edge 2: 1--3

Both vertices touch the edge.

degree[1] increases by 1 again
degree[3] increases by 1

Edge 3: 2--4

Both vertices touch the edge.

degree[2] increases by 1 again
degree[4] increases by 1

14.4 Final undirected graph result

Vertex	Degree	Explanation
1	2	Touches edges 1--2 and 1--3
2	2	Touches edges 1--2 and 2--4
3	1	Touches edge 1--3
4	1	Touches edge 2--4

15. Complete Phase 1 Validation Program

This version checks whether a directed graph has:

- self-loops
 - parallel edges
 - simple graph property
-

15.1 Full Code

```
#include <iostream>
#include <vector>
#include <set>
using namespace std;

int main() {
    int n = 4;

    vector<pair<int,int>> edges = {
        {1,2}, {1,3}, {2,3}, {3,3}, {1,2}
    };

    vector<int> indegree(n + 1, 0);
    vector<int> outdegree(n + 1, 0);

    bool hasSelfLoop = false;
    bool hasParallelEdge = false;

    set<pair<int,int>> seen;

    for (auto e : edges) {
        int u = e.first;
        int v = e.second;

        if (u == v) {
            hasSelfLoop = true;
        }

        if (seen.count({u, v})) {
            hasParallelEdge = true;
        }

        seen.insert({u, v});

        outdegree[u]++;
        indegree[v]++;
    }

    cout << "Directed graph degree report:\n";
    for (int v = 1; v <= n; v++) {
        cout << "Vertex " << v
            << " | indegree: " << indegree[v]
            << " | outdegree: " << outdegree[v] << "\n";
    }
}
```

```
    cout << "\nSelf-loop present? "
        << (hasSelfLoop ? "Yes" : "No") << "\n";

    cout << "Parallel edge present? "
        << (hasParallelEdge ? "Yes" : "No") << "\n";

    cout << "Simple graph? "
        << (!hasSelfLoop && !hasParallelEdge ? "Yes" : "No") << "\n";

    return 0;
}
```

16. Validation Program Explanation

The graph edges are:

```
1 -> 2
1 -> 3
2 -> 3
3 -> 3
1 -> 2
```

16.1 Self-loop check

This line checks for a self-loop:

```
if (u == v) {
    hasSelfLoop = true;
}
```

If the start and end vertices are the same, the edge is a self-loop.

Example:

```
3 -> 3
```

So the graph has a self-loop.

16.2 Parallel edge check

This line checks whether the edge has already appeared:

```
if (seen.count({u, v})) {  
    hasParallelEdge = true;  
}
```

The edge:

```
1 -> 2
```

appears twice.

So the graph has a parallel edge.

16.3 Simple graph result

A graph is simple only if:

```
No self-loop exists.  
No parallel edge exists.
```

This graph has both:

```
A self-loop: 3 -> 3  
A parallel edge: 1 -> 2 appears twice
```

So:

```
Simple graph? No
```

17. Time and Space Complexity of the Validation Program

The program uses a `set` to remember edges that have already appeared.

Time complexity

$O(E \log E)$

Why?

Because there are E edges, and each `set` operation takes about $\log E$ time.

Space complexity

$O(V + E)$

Why?

Because the program stores:

indegree array: $O(V)$
outdegree array: $O(V)$
seen edges set: $O(E)$

18. Common Beginner Mistakes

Mistake 1: Treating every graph as undirected

Wrong idea:

If 1 points to 2, then 2 must also point to 1.

Correct idea:

This is only true for undirected graphs.
In directed graphs, direction matters.

Mistake 2: Confusing indegree and outdegree

Wrong idea:

Indegree and outdegree are the same thing.

Correct idea:

Indegree = arrows entering a vertex.
Outdegree = arrows leaving a vertex.

Memory trick:

INdegree = arrows going IN
OUTdegree = arrows going OUT

Mistake 3: Thinking connected means strongly connected

Wrong idea:

If a directed graph looks connected, it is strongly connected.

Correct idea:

In a directed graph, you must be able to reach every vertex from every other vertex while following arrows.

Mistake 4: Forgetting what makes a graph simple

Wrong idea:

A graph is simple if it is easy to draw.

Correct idea:

A simple graph has no self-loops and no parallel edges.

Mistake 5: Thinking every path is a cycle

Wrong idea:

Any route through a graph is a cycle.

Correct idea:

A cycle must return to the starting vertex.

Mistake 6: Thinking adjacent means reachable

Wrong idea:

If I can reach vertex 3 from vertex 1, then 1 and 3 are adjacent.

Correct idea:

Adjacent means directly connected by one edge.
Reachable may require many edges.

Example:

1 ---- 2 ---- 3

1 can reach 3, but 1 and 3 are not adjacent.

19. Phase 1 Summary Table

Concept	Simple meaning
Graph	A structure made of vertices and edges
Vertex	A node, item, or object
Edge	A connection between vertices
Directed graph	A graph with arrows

Concept	Simple meaning
Undirected graph	A graph with two-way edges
Incoming edge	An edge entering a vertex
Outgoing edge	An edge leaving a vertex
Indegree	Number of incoming edges
Outdegree	Number of outgoing edges
Degree	Number of edges touching a vertex in an undirected graph
Adjacent vertices	Vertices directly connected by an edge
Self-loop	An edge from a vertex to itself
Parallel edges	Multiple edges between the same vertices
Simple graph	A graph with no self-loops and no parallel edges
Connected graph	A graph where every vertex can reach every other vertex
Non-connected graph	A graph split into separate pieces
Connected component	One connected piece of a graph
Articulation point	A vertex whose removal breaks the graph into more pieces
Biconnected graph	A connected graph with no articulation point
Strongly connected directed graph	A directed graph where every vertex can reach every other vertex by following arrows
Path	A valid route through connected vertices
Cycle / Circuit	A path that starts and ends at the same vertex
DAG	A directed graph with no cycles
Topological ordering	A valid linear order of vertices in a DAG

20. Mini Practice Questions

Use these questions to test your understanding.

Question 1

What are the vertices and edges in this graph?

```
1 ---- 2
|       |
3 ---- 4
```

Answer:

```
Vertices: {1, 2, 3, 4}
Edges: {1--2, 1--3, 2--4, 3--4}
```

Question 2

Is this graph directed or undirected?

```
1 ---> 2
2 ---> 3
```

Answer:

Directed, because the edges have arrows.

Question 3

Find the indegree and outdegree of vertex 1.

```
1 ---> 2
1 ---> 3
4 ---> 1
```

Answer:

```
indegree(1) = 1
outdegree(1) = 2
```

Question 4

Find the degree of vertex **1**.

```
1 ---- 2
|
3
```

Answer:

```
degree(1) = 2
```

Question 5

Are vertices **1** and **3** adjacent?

```
1 ---- 2 ---- 3
```

Answer:

```
No.
1 can reach 3 through 2, but 1 and 3 are not directly connected.
```

Question 6

Is this graph simple?

```
1 ---- 2
1 ---- 2
3 ---- 3
```

Answer:

```
No.
It has parallel edges between 1 and 2, and a self-loop at 3.
```

Question 7

How many connected components are here?

1 ---- 2 3 ---- 4 5

Answer:

3 connected components:
{1, 2}, {3, 4}, and {5}

Question 8

Is this a cycle?

1 -> 2 -> 3 -> 1

Answer:

Yes.
It starts at 1 and returns to 1.

Question 9

What does DAG mean?

Answer:

DAG = Directed Acyclic Graph

That means:

A graph with arrows and no cycles.

Question 10

Why can a graph with a directed cycle not have a valid topological ordering?

Answer:

Because a cycle creates an impossible dependency order.
Something would need to come before itself indirectly.

21. Final Phase 1 Checklist

You understand Phase 1 if you can explain these without looking:

- What a graph is
- What vertices are
- What edges are
- What $G = (V, E)$ means
- The difference between directed and undirected graphs
- What incoming and outgoing edges are
- What indegree and outdegree mean
- What degree means in an undirected graph
- What adjacent vertices are
- What self-loops are
- What parallel edges are
- What a simple graph is
- What connected and non-connected graphs are
- What connected components are
- What articulation points are
- What biconnected graphs are
- What strongly connected directed graphs are
- What paths are
- What cycles or circuits are
- What DAGs are
- What topological ordering means
- How the C++ degree-counting program works
- Why direction matters in directed graphs
- Why cycles prevent topological ordering

22. What Comes After Phase 1?

After Phase 1, you are ready for **Phase 2: Representing Graphs in Memory**.

In Phase 2, you will learn how to store graphs in C++ using:

- adjacency matrix
- adjacency list
- weighted graph representation
- compact list representation
- directed graph representation
- inverse adjacency list

Phase 1 gives you the vocabulary.

Phase 2 teaches you how to store that vocabulary in memory so algorithms can use it.